

Trabalho Final

Agente de Análise Exploratória de Dados
em Linguagem Natural

Construção de um Agente LLM com *Tool Use* sobre CSV

Tecnologia em Ciência de Dados | 5º semestre

Fatec Ourinhos

Agenda

- 1 O que vamos construir
- 2 Por que esse trabalho
- 3 Chatbot vs. Agente
- 4 Escopo técnico
- 5 Ferramentas obrigatórias
- 6 Restrições computacionais
- 7 Datasets disponíveis
- 8 Avaliação: o coração do trabalho
- 9 Entregáveis
- 10 Cronograma
- 11 Rubrica
- 12 Regras e perguntas frequentes

O que vamos construir

Pergunta orientadora

Dado um CSV qualquer e uma pergunta em linguagem natural

— “*qual a correlação entre idade e renda nos clientes do Sudeste?*” —

como construir um agente que **escolha sozinho** as operações pandas adequadas, **execute-as** e **interprete os resultados**?

O usuário pergunta:

“Existem outliers na coluna preço, separados por categoria de produto?”

O agente:

- Identifica colunas mencionadas
- Decide chamar `agrupar()` + `detectar_outliers()`
- Executa, trata erros
- Responde em português claro

Consolida o semestre

- Text mining
- Embeddings
- Transformers e LLMs
- Avaliação de modelos

Alinhado ao mercado

- *Agents* são o paradigma de 2024–2026
- *Tool use* é habilidade procurada

Habilidades exercitadas

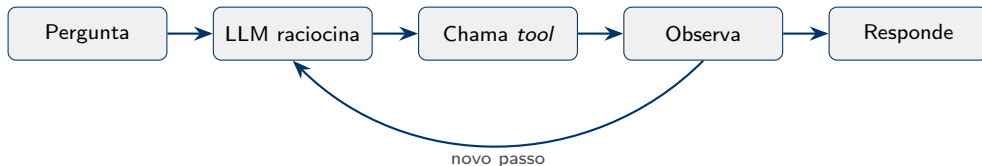
- Decomposição de problemas
- Engenharia de *prompts*
- Avaliação rigorosa (*benchmark*)
- Análise de trajetórias
- Discussão crítica

Chatbot tradicional

- Fluxo determinístico
- LLM gera **respostas**
- Não decide *o que fazer*
- Previsível, mas limitado

Agente

- Decide *quais tools* chamar
- Decide *em qual ordem*
- Encadeia múltiplos passos
- Trata erros e se adapta



O agente DEVE:

- Carregar um CSV
- Aceitar perguntas em português
- Escolher *tools* autonomamente
- Encadear chamadas múltiplas
- Tratar erros sem travar
- Gerar gráficos quando útil
- Registrar (logar) cada passo

O agente NÃO precisa:

- Treinar modelos preditivos
- Lidar com múltiplos CSVs
- Persistir memória entre sessões
- Ter interface web sofisticada

CLI ou Streamlit simples já basta.

Ferramentas obrigatórias (mínimo)

Ferramenta	O que retorna
<code>listar_colunas()</code>	Nomes e tipos de cada coluna
<code>descrever_dados()</code>	Estatísticas (describe), incluindo categóricas
<code>contar_valores(col)</code>	Distribuição de valores de uma coluna
<code>filtrar(condicao)</code>	Subconjunto com estatísticas
<code>agrupar_e_agregar(g, c, f)</code>	groupby + agg
<code>correlacao(a, b)</code>	Pearson ou Spearman
<code>detectar_outliers(col)</code>	Via IQR ou z-score
<code>gerar_grafico(tipo, cols)</code>	Salva imagem (hist, boxplot, scatter, barplot)

★ **Bônus na rubrica:** adicionar *tools* além das obrigatórias, com justificativa.

Regra de ouro

Este trabalho roda em **notebook comum, sem GPU**. Não será aceito trabalho que dependa de *fine-tuning* ou hardware acelerado.

Stack recomendada

- Python 3.10+
- pandas, numpy, matplotlib
- Streamlit ou CLI
- LangChain / Pydantic AI
(*opcional*)

LLMs aceitos (via API)

- Claude Haiku ou Sonnet
- GPT-4o-mini
- Gemini Flash
- Ollama local (Llama 3.2 3B)

\$ Custo estimado de API: **menos de US\$ 2 por grupo** no semestre inteiro.

Datasets disponíveis

- **Adult Income** (UCI) – renda e dados socioeconômicos
- **Titanic** (Kaggle) – sobrevivência
- **COVID-19 Brasil** – casos por município
- **ENEM** – microdados (amostra $\leq 50k$)
- **Vendas / e-commerce** (Kaggle)
- **Spotify Tracks** – características musicais

Requisitos mínimos

- Pelo menos **6 colunas** (3 numéricas + 2 categóricas)
- Pelo menos **1.000 registros**
- Outros datasets aceitos *mediante aprovação do professor*

Não basta implementar. É preciso medir.

Cada grupo entrega um `benchmark.json` com 30+ perguntas:

Tipo	Qtd.	Exemplo
Factual simples	10	<i>“Quantas linhas tem o dataset?”</i>
Analítica (multi-tool)	15	<i>“Qual a renda média dos homens com mais de 40 anos?”</i>
Ambígua/inválida	5	<i>“Qual a melhor coluna?”</i>

Cada pergunta vem com **resposta de referência** calculada manualmente em pandas.

Quantitativas

- Acurácia da resposta final (%)
- Taxa de execução bem-sucedida
- Nº médio de *tool calls*
- Latência média
- Custo médio por pergunta

Qualitativas

- 3+ trajetórias completas comentadas
- Análise de 3+ erros do agente
- Discussão sobre alucinação

Análise de trajetórias

Não basta dizer “acertou 70%”. É preciso olhar **como** o agente chegou na resposta: quantas *tools* chamou, em que ordem, e onde se perdeu.



Repositório Git

- Código modularizado
- README com instruções
- `requirements.txt`
- Dataset (ou link)
- `benchmark.json`
- Logs de execução



Relatório PDF

- Até 12 páginas
- Arquitetura (com diagrama)
- Resultados quantitativos
- Análise qualitativa
- Discussão crítica



Apresentação

- 15 min + 5 de perguntas
- Demonstração ao vivo
- Qualquer integrante pode ser questionado

Cronograma sugerido (8 semanas)

Semana	Atividade
1	Formação dos grupos, leitura do enunciado, escolha do dataset e LLM
2	Análise exploratória “manual” do dataset. Definição das <i>tools</i>
3	Implementação das <i>tools</i> em Python puro. Testes unitários
4	Integração com a API do LLM. Primeiro fluxo <i>end-to-end</i>
5	Construção do <code>benchmark.json</code> . Refinamento do agente
6	Execução do benchmark. Coleta de métricas. Análise de erros
7	Redação do relatório. Preparação da apresentação
8	Apresentações

Dimensão	Peso
Definição do escopo e do dataset	10%
Design do agente (<i>tools</i> , arquitetura, fluxo de decisão)	20%
Implementação (qualidade, modularização, tratamento de erros)	20%
Conjunto de avaliação bem construído	15%
Resultados quantitativos + análise de trajetórias	15%
Análise crítica (custo, latência, falhas, alucinação, ética)	10%
Demonstração funcional e relatório	10%
Total	100%

⇒ *Implementação + design valem 40% — é onde se faz a diferença.*

O que separa um trabalho excelente

Nível	Descrição
Insuficiente	Agente não funciona ou só em casos triviais. Benchmark ausente.
Regular	Funciona em perguntas simples; falha em multi-tool. Benchmark mínimo.
Bom	Funciona consistentemente. Benchmark bem construído. Análise de erros presente.
Excelente	Agente robusto, com <i>tools</i> extras. Benchmark rico. Análise crítica madura: alucinação, custo, ética.

O diferencial do trabalho “excelente”

Não é mais código. É **mais reflexão** sobre o que o código faz, onde falha e por quê.

Uso de LLMs durante o desenvolvimento

Permitido: usar ChatGPT, Claude, Gemini como assistentes de programação.

NÃO permitido: entregar código que o grupo não compreenda.

△ Durante a apresentação, qualquer integrante pode ser questionado sobre qualquer linha do código.

Cópia de código entre grupos resulta em nota zero para todos os envolvidos.

Posso usar LangChain/LlamaIndex?

Sim, desde que compreenda o que o framework faz por baixo.

E se o LLM “alucinar” uma coluna que não existe?

Esse é o tipo de comportamento que sua avaliação deve capturar. O agente deve idealmente recusar.

Posso usar Ollama localmente?

Sim. Penalidade: latência alta e raciocínio multi-passo inferior. Use para desenvolvimento, API para avaliação final.

Posso fazer fine-tuning?

Não. O escopo é construção de agente, não treinamento de modelo.

E se meu grupo tiver dificuldade técnica?

Procure o professor cedo. Travar na semana 7 é muito diferente de travar na semana 3.

Próximos passos (esta semana)

1. **Formar grupos** (2 a 4 alunos)
2. **Ler o enunciado completo** (PDF disponibilizado)
3. **Escolher o dataset** e validar com o professor
4. **Criar contas** nos provedores de LLM (créditos grátis)
5. **Inicializar o repositório Git** e compartilhar com o professor

Dúvidas técnicas no canal da disciplina. Dúvidas de escopo, na próxima aula.

Perguntas?

Bom trabalho.

Fatec Ourinhos | Ciência de Dados | 5º Semestre